1.05.01 - OO Modeling Practices.

Conceptual Modeling.-

Conceptual Modeling consists of representing ideas, concepts and characteristics of a system in an organized way to better understand it.
What is it for?

- Understand how a real system works.
- Represent how the system currently is and how it is desired to be.
- Document important decisions.
- Design a solution and plan its contruction.
- Comunicate ideas among team members.

UML and its diagrams.-

UML (Unified Modeling Language) is a modeling language that allows visually representing the structure and behavior of a system.
The main diagrams mentioned in the document are:

- Use case diagrams: represent who uses the system and what goals they need to archieve.
- Class diagrams: represent classes, their characteristics, and the relationships between them.
- Interaction diagrams: show how objects communicate with each other.
- State diagrams: describe the internal behavior of objects or methods.

Use clear names:

Names of classes, variables, and methods must comunicate their purpose and facilitate code reading and maintenance.

Element	How to name it	Example:
Class	Noun, usualy in PascalCase	StudentRecord
Variable	According to the object or its purpose, usually in lower CamelCase.	myStudent
Method	Verb or verb phrase.	computeCost()

1.05.01 - OO Modeling Practices.

Conceptual Modeling.-

Conceptual Modeling consists of representing ideas, concepts and characteristics of a system in an organized way to better understand it.
What is it for?

- Understand how a real system works.
- Represent how the system currently is and how it is desired to be.
- Document important decisions.
- Design a solution and plan its contruction
- Comunicate ideas among team members.

UML and its diagrams.-

UML (Unified Modeling Language) is a modeling language that allows visually representing the structure and behavior of a system.
The main diagrams mentioned in the document are:

- Use case diagrams: represent who uses the system and what goals they need to archieve.
- Class diagrams: represent classes, their characteristics, and the relationships between them.
- Interaction diagrams: show how objects communicate with each other.
- State diagrams: describe the internal behavior of objects or methods.

Use clear names:

Names of classes, variables, and methods must comunicate their purpose and facilitate code reading and maintenance.

Element	How to name it	Example:
Class	Noun, usualy in PascalCase	StudentRecord
Variable	According to the object or its purpose, usually in lower CamelCase.	myStudent
Method	Verb or verb phrase.	computeCost()

Perform unit testing.

Unit tests allow testing parts of the code individually, such as methods or classes. Their benefits are:

- Detect design errors.
- Verify that the code works correctly.
- Document how a method or class is used
- Ensure that changes do not break existing functionality through regression testing.

Main idea of the slides

Develop organized, easy-to-understand, flexible, and maintainable software through good modeling, low coupling, meaningful name, and automated testing.

1.05.07 - Introduction To UML

1. What is UML?

UML (Unified Modeling Language) is an object-oriented modeling language used to visualize, specify, and document software system. Key traits: expressive, semi-formal, process-independent, extensible, and supports incremental development.

Core Elements of UML.

UML elements serve as building blocks grouped into:

- Structural: Classes, objects, interfaces, components.
- Behavioral: Messages, states, transitions, events.
- Organizational & Extension: Packages, views, comments, stereotypes.

Modeling Perspectives

- Analysis: Understands real-world domain and problem.
- Specification / High-Level Desing: Defines system boundaries and requirements.
- Implementation: Focuses on code architecture and concrete software components

Types of UML Diagrams:

Diagrams are divided into two main categories:

Structural Diagrams:

- Class Diagrams
- Object Diagrams
- Component Diagrams
- Deployment Diagrams

Behavioral Diagrams:

- Use Case Diagrams
- Sequence Diagrams
- Communication Diagrams.
- State Diagrams
- Activity Diagrams.

∴ UML helps design, document, and communicate software structure and behavior effectively. Diagrams should add value, not just be visual drawings.

1.05.08 - Basic Use Cases.

What are Actors?

Actors are external entities (users, external software system, or hardware devices) that interact with the system to archive specific goals.

What are Use Cases?

A use cases represents a user goal achived through system functionality, independent of implementation details.

Use Cases vs System Interactions.

Aspect	Use Cases	System Interaction.
Focus	Answers "What?" "Why" (Goals and required functionality)	Answers "How" (Interaction steps from user perspective)
Example	Make a Purchase	Selec product → Add to cart → Review total → Confirm order

DD MM AA

Use Case Diagrams.

Visual representations of actor goals and system compabilities. Core components: Actors, Use Cases, Relationships, and Subjects.

Benefits of Use Cases

Identify visible functions, map permissions, promote reuse, guide incremental planning, and structure test cases

Main Takeaway: Use cases define what actors need to achieve deciding how to build implementation details.

1.05.09 - Class Diagrams. - Objects, Classes, and Basic Class Diagrams.

What is an Object?

An object represents a specific entity with boundaries, state, behavior, and unique identity (e.g. a student with a name, major, and GPA).

What is a Class?

A Class is an abstraction defining common characteristics for a group of objects.

- Key Distinction: A class is the blueprint, an object is a specific instance.

Class perspectives

- Analysis: Real-world domain concepts.
- Specification: Interfaces and system contracts.
- Implementation: Code structures in programming languages.

Class structure:

A Class represented as a rectangle divided into three compartaments:

1. Name: Class identifier (e.g., Student).
2. Attributes: Data properties (name: String).
3. Operations: Methods/services provided (addCSection()).

-Class Naming Rules

- Singular nouns in PascalCase / UpperCamelCase (StudentRecord)
- Meaningful to the domain; avoid purely technical or implementation-specific names.

Atributes and Visibility

Atributes store values representing objects properties.

- UML Syntax: [visibility] name [multiplicity] [:type] [= initial value]

Symbol	Visibility
+	Public (public)
#	Protected (protected)
-	Privoite (private).

Operations.

Operations represent actions or services (member functions / methods) provided by an object (addCourse(). calculateTotal()).

Attributes = Characteristics / data

Operations = actions / methods